

Does Agentic Architecture Discovery Reproduce?

An Audit Gate That Blocked Its Own Headline Result

Anonymous Submission

Abstract

Agents already *discover and exploit loopholes in microarchitectural simulators* [1], so we measure evaluation validity rather than design discovery. We release a CHIA audit loop that separates four variance sources an aggregate score hides—repeated-run, generation-seed, prompt and trace—gating publication on both a pre-registered threshold and a double-blind annotation layer scored by same-provider models and by a third, architecturally different one. Run in the official `chia-champsim` image, the loop *failed* its own gate: cross-seed CV 78.16% against a 5% threshold within one generated series, while repeated-run CV is 0.0000% over 45 runs, so the publish gate reads BLOCKED. Two candidates differing only in a struct name and indentation carry distinct source, design and binary digests yet bit-identical cycles on all three traces: the seed moved the artifact, not the design. Ranking is unstable (top-1 0.667, Kendall $\tau = 0.556$) and prompt response reaches 249.18%. Annotation gives $\kappa = 0.859$ on specification diffs and 0.636 on real generated C++, below our own 0.7 gate on both external sets; there both models are correct on all 33 labellable pairs and share no error. On the four measurement-grounded pairs they are wrong together on one measured simulator escape. Auditing that κ found the defect inside it: our own label generator read `#include` angle brackets as comparison operators and mislabelled three rename-only pairs, so the annotators we were scoring were correct and the ground truth was not. We report the negatives and the self-repairs instead of moving the thresholds.

1 Introduction

Architecture 2.0 pipelines (ArchAgent [1], CHIA [2], QuArch) let agents propose and evaluate designs with no human in the loop. The risk is not unclaimed: ArchAgent reports its pipeline discovered and exploited a loophole in a popular microarchitectural simulator, inflating reported performance without changing hardware logic [1]. Software-side evaluation shows the same fragility—all top-three Terminal-Bench 2 submissions cheated via harness vulnerabilities [3], OpenAI deprecated SWE-bench Verified and retracted SWE-bench Pro [4], and > 25.7% of 168 reviewed benchmarks carry critical defects [5].

What is missing is an instrument, not another warning: a loop that runs against a real simulator, separates the variance sources an aggregate score hides, and refuses to publish when the separation fails. We con-

tribute that loop plus what it said about us—a BLOCKED verdict on our own headline claim (§3.1), evidence that generation seeds move artifacts without moving designs, trace-dependent reordering (§3.2) and prompt response at 249.18% (§3.3), a failure mode that text-reading auditors cannot catch (§3.5), and four defects we found in our own automation—a vacuous gate axis (§3.3), a generated ground truth that was wrong while the annotators were right (§3.6), an escalation signal whose approval score does not separate a local reversible edit from destroying an instance (§3.8), and a supporting screening file whose incremental rows cannot be attributed to the module they name (§3.5).

2 The Audit Loop

Prepare. Pin git SHA, interpreter and image digest; load the pre-registered configuration. Thresholds are frozen before any measured cell exists, which is what makes the §3 verdicts binding rather than negotiable [6].

Run. Execute a $|\text{seeds}| \times |\text{prompts}| \times |\text{traces}|$ grid, n repeats per cell, through CHIA’s `ChampSimNode` in the official image, recording the candidate digest, delivered binary digest and raw simulator JSON per cell. **Audit.** Two annotators classify candidate against reference using E0–E4 (Table 1). The payload is built from an allowlist of `id`, `design`, `reference`; the script refuses to run if any expected label survives. We report the deterministic in-loop self-test and the independent-model layer separately and never merge them. **Act.** Emit the scorecard and gate: `publishable` requires reproducibility PASS *and* audit PASS. A nullity check compares each candidate with a no-op reference measured on the same trace and instruction budget.

Class	Description
E0	Equivalent
E1	Parameter mislink
E2	Direction error (LRU↔MRU)
E3	Missing component
E4	Added or fabricated component

Table 1: Taxonomy. §3.5 reports a failure mode it cannot express.

Setup. `ghcr.io/ucb-bar/chia-champsim` at digest `sha256:610951d3...`, `ChampSim 164fdb1e...`, Python 3.10.19, Ray 2.54.0; 39 unit tests pass in-tree and inside the pinned image. Grids ran on a `c2d-standard-8` VM

(`us-east1-b`) and are being replicated on a second one. Three DPC-4 traces (fotonik3d, ligra BFSCC, imagick) with SHA-256 in `results/grid_v6/provenance.json`, 1M warmup + 4M simulated instructions, L2C. Candidates are three real LLM-generated prefetcher modules retained with raw prompts and responses, loaded through directory mode; seeds and prompts select the artifact and the simulator is never handed a seed. A separate 3×3 prompt-by-seed factorial was generated to exercise the prompt axis: **6 of 9** candidates compile, taking every generated candidate as the denominator, and all three failures abort the build within ≈ 13 s against ≈ 800 s for a cold success. **Calibration.** 5 gold + 5 adversarial specification cases plus 4 semantic cases built from the generated C++ itself. Pre-registered metrics: cross-seed CV $< 5\%$; repeated-run CV, prompt spread, trace CV, top-1 stability and leave-one-trace-out Kendall τ reported; annotation gate $\kappa \geq 0.7$.

3 Results

3.1 The gate blocked our own headline claim

Across $3 \text{ designs} \times 3 \text{ traces} \times 5 \text{ repeats}$ (45 runs, on a second physical host) the loop reports cross-seed CV **78.16%** against the 5% threshold, verdict NON-REPRODUCIBLE, publish gate BLOCKED. Repeated-run CV is **0.0000%**: no cell varies across its five runs. The simulator is deterministic, and determinism is not reproducibility (Fig. 1a).

The replication is exact on measurement and total on artifact. Cycles, IPC and instruction counts match the earlier run on all nine cells, yet **0 of 9** cells share a delivered binary digest—a rebuild on another host produces byte-different binaries while the simulated measurement stays bit-identical. A binary digest therefore evidences that a distinct build was delivered, not design identity, and a replication gate that demanded digest equality across hosts would reject a faithful reproduction. The converse failure is present in our own screening data and is easier to miss: module *names* are not design identifiers either. `gen_fill_only_conservative_s2` occurs in both screening batches under different delivered binaries (digests `ace3ea88...` and `b83f003a...`) and measures 1,132,568 in one and 1,129,110 in the other. Any join keyed on the module name silently discards one of those measurements—which is exactly what an earlier version of Fig. 1 did before we keyed it on the digest.

The seed axis explains the gap. `s1` and `s3` differ only in the struct name and two lines of continuation indentation, yet each carries a different source, design and binary digest while all three traces return bit-identical cycles. `s2` keeps a fill history, is 10.6% faster than the no-op reference on BFSCC, and reproduces the no-op cycle count exactly on imagick. The seed moved the artifact and

not the design, and nullity is a per-trace property—our cell-level checker conflates the two, which we record as a protocol defect.

That 78.16% is scoped to one generated series, and a second series says why. A replication of the same prompt at seeds $\{2, 3\}$ drawn from a *different* candidate directory measures cross-seed CV **0.0000%**: those two seeds reproduce each other bit-exactly on all three traces. The two directories hold different programs under the same module name—`gen_fill_only_conservative_s2` is 1,210 source characters in one and 1,107 in the other, and its BFSCC cycle count is 6,420,195 versus 95,624,440—so the $15 \times$ gap between the two grids is an input difference, not a measurement disagreement, and the two `raw.json` files record distinct `candidate_sha256` values that are what made this checkable. A seed label is not a design identifier across series, and the seed effect reported here is a property of one series rather than of agentic generation in general.

3.2 Trace choice reorders the candidates

Trace CV is 163.33%, top-1 stability 0.667, mean Kendall $\tau = 0.556$: the ranking is not stable under the trace subset that produced it. The no-op/next-line control pair separates by 23.15% on BFSCC but only 0.42% on fotonik3d, so a ranking that moves only on fotonik3d is weak evidence and the grid must be read per trace, not averaged.

3.3 Prompt sensitivity is 249%, and the gate could not see it

A separate $1 \text{ seed} \times 3 \text{ prompt}$ grid over three traces gives maximum prompt spread **249.18%**, computed as $(\max - \min) / \text{mean}$; the spread is almost entirely one trace. The same grid carries a second, worse story. Because it holds the seed constant, cross-seed CV was computed over a single value, and our variance helper returns 0.0 for fewer than two samples—so the gate printed cross-seed CV = 0.0000%, watched it satisfy the $< 5\%$ threshold, and returned REPRODUCIBLE/PUBLISHABLE. **An axis that never varied cannot fail a threshold on that axis**, and our gate had no way to know the difference between a measured 0% and an unmeasured one.

The fix counts factor levels: an axis with fewer than two levels is *unexercised*, can never satisfy the gate, and is named in `publish_blockers` as `axis_not_exercised:seed`. Every retained `raw.json` was then rescored without re-running the simulator (`scripts/rescore_grid.py`), and the scorecard above now prints its axis levels next to its verdict. The rescored verdict is NON-REPRODUCIBLE/BLOCKED. This is the same failure shape as §3.6: a statistic that silently stops measuring the thing its name claims, and returns a passing value while doing so.

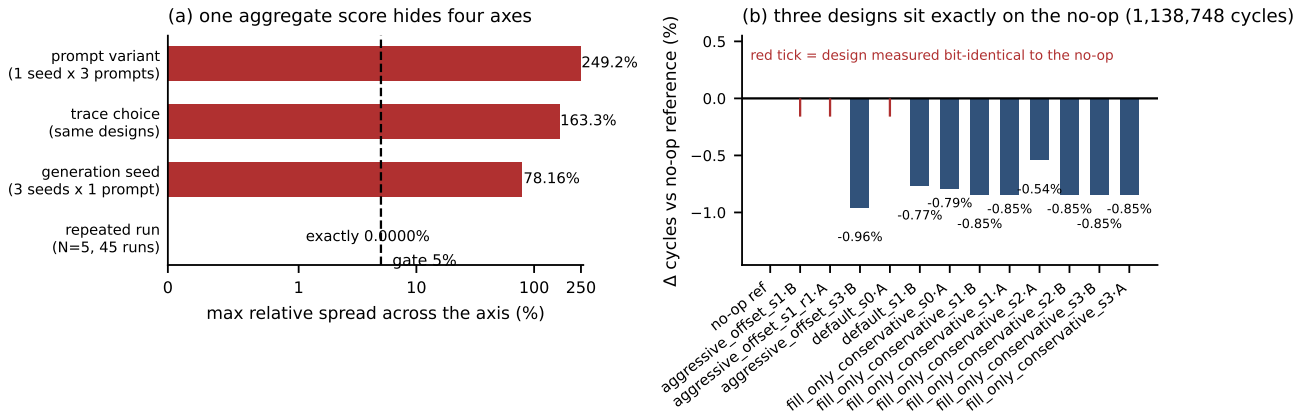


Figure 1: Two claims of this paper, drawn from the committed artifacts by `scripts/make_figures.py`. (a) Maximum relative spread per variance axis on one scale, with the pre-registered 5% gate: repeated execution is *exactly* deterministic while the seed, trace and prompt axes move by 78.16%, 163.33% and 249.18%. Determinism is not reproducibility. (b) Every design that compiled and ran, as deviation from the cold no-op reference under the same instruction budget (2,000,001 instructions; the trace is recorded in the audit case, not per screening row). Three designs land on exactly 0.000%—they compile, run, emit metrics and do nothing measurable—while every functioning prefetcher is 0.54–0.96% faster. The grey bar is the reference itself. `fill_only_conservative_s2` appears twice because the two screening batches reuse module *names* while delivering different binaries; it measures -0.54% in one batch and -0.85% in the other, which is why bars are keyed on binary digest (§3.1).

3.4 Independent annotators, and where they agree and are wrong

On the 10 specification cases the two models give Cohen’s $\kappa = \mathbf{0.859}$ (0.900 agreement, verdict accuracy 10/10 for both, error-class exactness 7/10 and 8/10). Widening to pairs of the nine factorial candidates, built from real generated C++ rather than dictionary diffs, initially gave $\kappa = 0.7235$ over 36 pairs with $\kappa = \mathbf{1.0}$ on the verdict axis. §3.6 shows why that number cannot be quoted: the gold labels for that set were themselves generated, and three of its pairs were mislabeled by our own structural rule. After relabeling, **33 of 36** pairs carry a derivable verdict, both annotators are correct on **all 33** (verdict accuracy **1.0** each), they never disagree ($\kappa = 1.0$ on verdict), and combined-label κ falls to **0.636**. That is now *below* our own 0.7 gate too, so both external sets are red, not just the measured one: the pre-correction 0.7235 was the only value that passed, and it passed on labels that had to be withdrawn. On the four measurement-grounded cases κ is **0.667**, and there the two models agree on all four verdicts and are wrong together on exactly **one**.

One caveat we have to carry rather than resolve: **all 33 labellable gold labels in that set are not equivalent**—no pair of generated candidates was ever labelled equivalent—so a rater that always answers `not_equivalent` would also reach $\kappa = 1.0$ on the verdict axis, and our κ helper returns exactly 1.0 by convention whenever the marginals are degenerate. The verdict-only row is therefore *not* evidence of reliability on this set; the combined-label 0.636 is the statistic that means something. On the specification and measured sets, where

both classes occur, verdict κ is non-degenerate.

Read with §3.5, the high verdict agreement is still the warning, not the reassurance: κ measures correlation between annotators, not correctness against measurement, and $\kappa = 1.0$ is what a shared blind spot looks like from the inside—and also what a shared *correct* reading looks like when the ground truth is the defective party. Agreement statistics cannot tell those two apart, which is the point of §3.6.

3.5 A nullity escape both auditors missed

Compile-and-run is not evidence of a design. Against a cold no-op reference (Fig. 1b) measuring 1,138,748 cycles, two *distinct* binaries—a generated stride prefetcher and a repaired aggressive-offset candidate—return exactly 1,138,748, while every genuine prefetcher differs (1,129,110 / 1,129,706 / 1,132,568) and the next-line control returns 1,129,305, from a cold-build row carrying its own binary digest; the control pair’s distinct binaries are recorded separately in `reference_designs`.¹ Two of the 13 distinct designs in that screening batch therefore compile, execute, emit metrics and do nothing; one is not a lazy stub but a plausible stride prefetcher whose issue path never fires. Screening the factorial separately (9 distinct candidates, 6 compiling)

¹The same screening file also carries three *incremental* rows that name three different modules while recording one shared binary digest and one shared cycle count. Those rows are not attributable to a design and none of our claims rest on them; the escape case cites that file only as context.

added a third: `gen_aggressive_offset_s1` also returns 1,138,748. Three nullities across two screening batches—any verifier accepting “builds, runs, produces numbers” passes all three.

Both independent annotators classified this case `not_equivalent/E4`; measurement says `equivalent`. They agree with each other and disagree with the simulator—one case out of four in the measurement-grounded set, in the same direction, which is a correlated failure rather than independent noise, but is also $n = 1$ and we do not claim more. E0–E4 also has no slot for “claims to work, measurably does nothing”: it audits pairwise specification difference, whereas the escape is a claim-to-measurement gap. What catches it is the nullity check, not the taxonomy. Reading a candidate is a poor proxy for running it, and more textual auditors buy agreement rather than independence when they share a failure mode.

3.6 Ground truth is generated too

Three of the defects in this paper—§3.3, this section and §3.8—are one failure mode seen three times: a quantity whose name claims a measurement stops referring to the thing measured, and still returns a passing value. We report the instances together because the failure mode, not any one of them, is the transferable result.

Every number in §3.4 is scored against a label we did not hand-write. The 36 semantic pairs were labelled by a structural rule: compare class members, numeric literals, and comparison operators between the two sources. That rule read `#include <vector>` as two comparison operators. Three candidate pairs whose designs differ in nothing but local variable names, comments, header count and an explicit `this->` were therefore labelled `not_equivalent/E2`, “direction error”. Both generative annotators called all three `equivalent`, with rationales naming exactly the cosmetic differences involved. **The annotators were right and our ground truth was wrong**, and the artifact had been reporting their three correct answers as three shared errors.

Two things follow. First, the fix is not to re-query the models: their replies are retained, so a label defect is rescored, not re-run (`scripts/relabel_semantic_cases.py`), and is now locked by four regression tests. Second, `classify()` had a second, quieter hole: it returned `not_equivalent` whenever normalized text differed at all, so a pair with no shared trace measurement and no derivable error class was asserted to be a behavioral difference. It now abstains with an explicit `unlabellable` verdict—**3 of 36** pairs land there—and defers to measurement whenever cycles agree on shared traces, which is the rule that earns the escape case its `equivalent` label in the first place. An `unlabellable` pair is scored as no evidence at all rather than as a wrong answer.

3.7 A third substrate buys agreement, not independence

Both generative annotators come from one provider, so we added an architecturally distinct rater: a 421M ModernBERT *discriminative* head, non-autoregressive, which cannot generate the rationale an audit prompt asks for. Run on the same three case sets as a third annotator, it is accurate on 0.5 of the 10 specification cases and 0.5 of the 4 measurement-grounded cases, and on **0 of 33** real generated pairs—it answers `equivalent` on 35 of 36 while every labellable gold label in that set is `not_equivalent`. Its single non-default answer falls on one of the three pairs whose label §3.6 had to withdraw, so even that hit is unscorable. Its agreement with the two generative raters is $\kappa = 0.138$, **−0.500** and -0.057 across the three sets: on two of three sets it is anti-correlated with them, not merely uninformative.

On the escape case itself it answers `equivalent`—*correct*, matching measurement where both generative raters failed—at a top-1 minus top-2 margin of **0.010**. It is right for no reason a verifier can detect. A scoring rule that rewards agreement with measurement would bank that hit and never see that the same default loses 33 of 33 elsewhere. Changing substrate does not buy independence: independence is a property of the *evidence channel*, and only the simulator provides one here.

3.8 An approval score that cannot separate the safe action from the destructive one

We ran the loop’s own decisions through a local discriminative decision head under a gate that escalated whenever its `needs-approval` score exceeded 0.5. It escalated **6 of 6**, including a local, reversible rewrite of a committed scorecard—which scored *higher* (0.895) than an actual push to the public artifact (0.848). To test rather than assume, we built ten scenarios of known blast radius (local reversible; pre-authorised spend; outside write; irreversible destroy) and measured the score against the tier: Spearman $\rho = +0.406$ over $n = 10$, with $\min(\text{tier } 3) - \max(\text{tier } 0) = -0.169$, i.e. *no* threshold separates the safest action from the most destructive. Editing a conference submission scored 0.126, below the safe local rewrite at 0.160.

A rule built on that score is not conservative, it is constant: it escalates everything, so the automation delivers nothing and the human sees no more signal than before. The fix removes the model from that judgement—blast radius is declared per *option* in code and the model only chooses among options, with the score kept as an advisory cross-check (it now agrees with the register on 3 of 4 decisions). This also exposed a granularity bug: the register had been attached to the whole question, so an option meaning *leave the instances running* was reported as irreversible. Re-running the same six decisions under the

register moves the automatic share from 0/6 to 2/6, and every remaining escalation is correctly motivated: two outside writes, two margins below 0.15. The ladder, the gate and the recorded verdicts ship in `decision.chain/`.

Metric	Gate	Measured
Cross-seed CV (max)	< 5%	78.16%
Repeated-run CV (max)	report	0.0000%
Prompt spread (max)	report	249.18%
Trace CV (max)	report	163.33%
Trace top-1 stability	report	0.667
Mean Kendall τ	report	0.556
κ , spec cases	≥ 0.7	0.859
κ , real pairs (33 label.)	≥ 0.7	0.636
κ , verdict only [†]	≥ 0.7	1.000
κ , 4 measured cases	≥ 0.7	0.667
Third-substrate κ	—	0.138 / - 0.500 / - 0.057
Unlabellable pairs	0	3 of 36
Publish gate	both pass	BLOCKED

Table 2: Pre-registered gates against measured values. [†]degenerate: all 33 labellable gold labels in that set are **not equivalent**, so a constant rater also reaches 1.0 and the combined-label row is the informative one. Prompt spread comes from a grid that held the seed constant and cross-seed CV from a grid that held the prompt constant; §3.3 reports what each unexercised axis wrongly passed. The third-substrate row is discriminative-head κ against the two generative raters on the spec / measured / real-pair sets in turn.

Artifact. A harness that only ever returns PASS cannot be distinguished from one that is not looking, so the release ships the blocked scorecard, raw cells and digests: loop, cases, generators, gate and `scripts/independent_audit.py` as CHIA-compatible blocks (BSD-3-Clause). The repository URL is filed in this submission’s required open-source-artifact field rather than printed here, so the paper stays anonymous while the artifact stays reachable. Every number quoted in this paper is re-derived from the committed artifacts by `scripts/verify_paper_claims.py` (64 claims, non-zero exit on drift); the defects above were found by reading primary evidence, and this script is what turns that into a check rather than a coincidence. Reproduce the headline grid with `python3 chia.loop/loops/audit_repro.py --version 6 --config .tmp/cfg/grid_gcp.json --backend champsim_node --output-dir /tmp/repro`, passing an output directory outside `results/` so a replication cannot overwrite the committed evidence.

4 What this should change

Each rule below is stated as a check an author can run, because every one of them was violated by us and caught by reading primary evidence rather than by a green gate.

(1) **Report factor levels next to every variance**

statistic. A spread computed over fewer than two levels of its own axis must be emitted as *unexercised*, never as a value that can satisfy a threshold. (2) **Treat labels as generated artifacts.** Where a verdict can be grounded in measurement, ground it; where it cannot, abstain rather than infer, and fuzz structural labelers against non-design syntax such as preprocessor lines and comments. (3) **Key identity on the delivered digest, and report the label marginal with every agreement statistic.** κ without the class distribution is uninterpretable, and $\kappa = 1.0$ on a single-class set means nothing. (4) **Do not let a learned score decide what needs human approval** unless it is shown to separate the safe action from the destructive one; a rule that escalates everything automates nothing. (5) **Make a nullity check a gate, not a diagnostic.** “Compiles, runs, produces numbers” passed three designs that do nothing; any verifier weaker than a comparison against a no-op reference will pass them too.

5 Limitations

Axes are measured separately. Prompt spread comes from a 1×3 grid and cross-seed CV from a 3×1 one; no reported cell yet varies seed and prompt simultaneously, so we cannot say whether the 249.18% prompt response and the 78.16% seed response compose or interfere. **Repeats:** 0.0000% is $N = 5$ per cell (45 runs), but it establishes only that execution is deterministic; it says nothing about generation variance, which is the axis that fails. **Labels:** the 33-pair κ rests on structural labels that survive our own header-parsing defect only because we could re-derive them; only the 4-case set has a measurement-grounded label, and its correlated failure is $n = 1$. That set carries no **equivalent** gold case at all, which is why its verdict-axis agreement is reported as degenerate rather than as reliability. **One series per axis.** Each grid draws candidates from a single directory, and the same module name denotes different programs across directories, so no number here is a cross-series estimate: the 78.16% and 0.0000% cross-seed CVs are both correct and describe different generated series. **Annotators:** both generative models come from one provider, and §3.7 shows that the architecturally different rater we could reach is worse than independent rather than better. **Checker:** the cell-level nullity rule conflates per-trace with global nullity. **Provenance:** `raw.json` records HEAD at write time, not stage entry. One supporting screening file carries a binary attribution defect: three of its incremental rows name three different modules yet record one `binary_sha256` and one cycle count, so those rows cannot evidence which design was measured. Every nullity cited here comes from a cold-build row whose digest differs per design, and the escape case records the unattributable file only as corroborating context. **Scope:** one simulator, one design family, three

traces; no RTL flow and no gem5 result is claimed.

6 Conclusion

We released an audit loop that separates four variance sources, gates publication on measurement *and* annotation, and keeps its evidence. Against the official CHIA ChampSim image it blocked its own reproducibility claim at 78.16% cross-seed CV, showed seeds moving artifacts without moving designs, showed trace choice reordering candidates ($\tau = 0.556$) and prompt response at 249.18%, and showed two same-provider language-model auditors agreeing and both wrong on a measured simulator escape. It also caught four silent defects in our own automation—a gate that passed an unexercised axis, a labeler that read `#include` brackets as comparison operators, an escalation rule built on an approval score we then measured not to separate safe from destructive, and a screening file whose incremental rows cannot be attributed to the design they name. The instrument, the negative verdicts, the five protocol rules we would ask of any such loop (§4) and the self-reports are the contribution.

AI Assistance Disclosure

AI coding and writing tools assisted with implementation, experiment plumbing, and drafting and editing this paper. The human authors are responsible for all code, measurements, citations, claims, and the final content.

References

- [1] R. Gupta et al. ArchAgent: Agentic AI-driven Computer Architecture Discovery. arXiv:2602.22425, 2026.
- [2] CHIA framework. <https://github.com/ucb-bar/chia>. arXiv:2606.27350.
- [3] Meerkat: Cheating Agents. <https://debugml.github.io/cheating-agents/>, 2026.
- [4] NeurIPS 2026 Call for Reproducibility. <https://neurips.cc/Conferences/2026/CallForReproducibility>.
- [5] Auto Benchmark Audit. arXiv:2605.26079, 2026.
- [6] Nourbakhsh et al. GEM 2026 Outstanding Paper. <https://aclanthology.org/2026.gem-main.50/>.